IT2080 IT Project
Year 2, Semester 2, 2025
Activity 04 - Database Design & Development Activity

TNR STEEL ERP System

Group Details

Campus: Malabe Campus

ITP Group Number: ITP25_B1_14

IT Number	Student Name	Email	Contact Number
IT23566798	Wijeratne W M D N	it23566798@my.sliit.lk	0753393063
IT23543164	Jeewantha R M C	it23543164@my.sliit.lk	0743870194
IT23562974	Induwara K A D M	it23562974@my.sliit.lk	0703976763
IT23567788	Ranmith G A K	it23567788@my.sliit.lk	0778966083
IT23565180	Francisco C T	it23565180@my.sliit.lk	0742885255

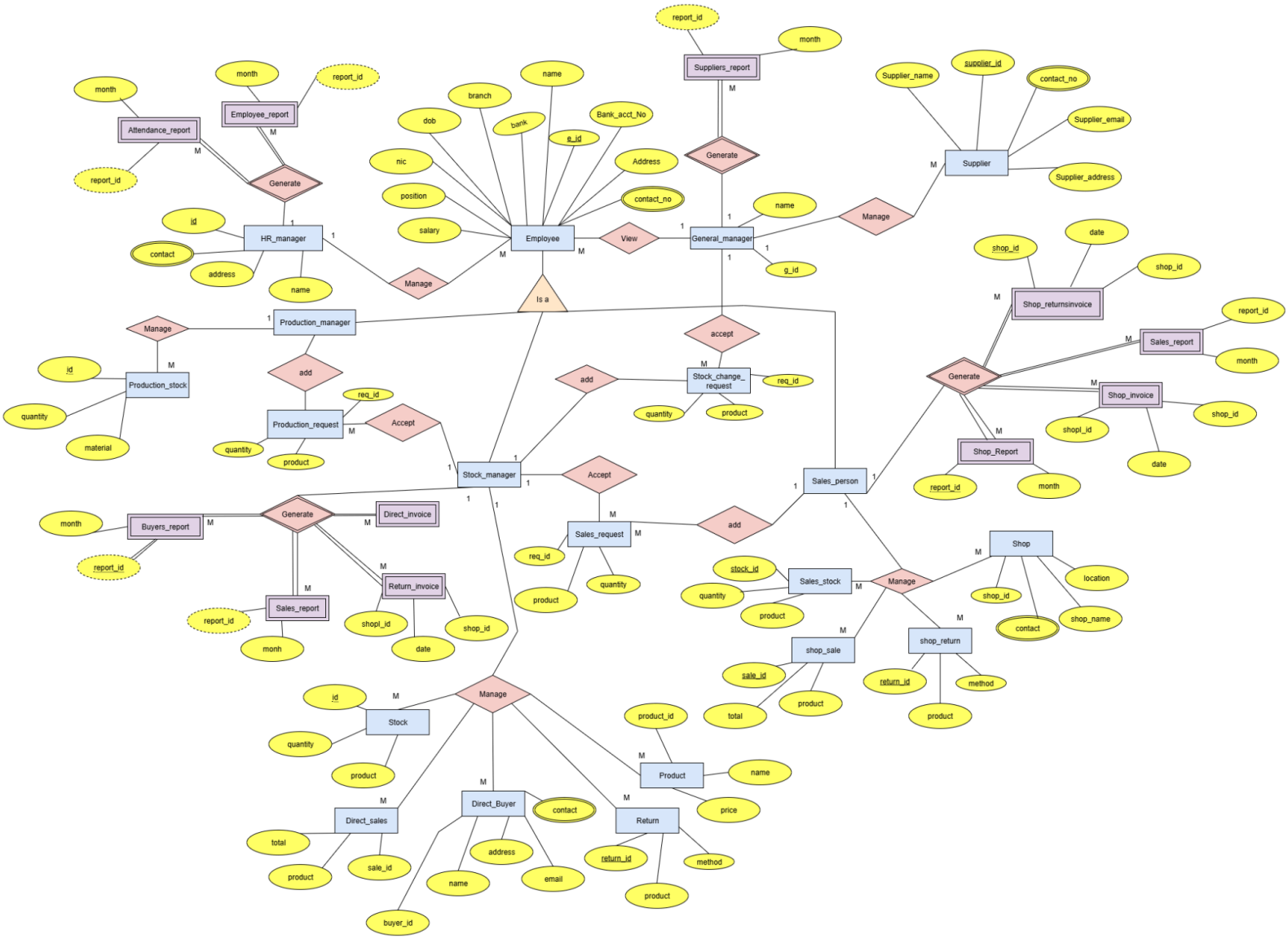
Contents

1.ER diagram 3

2. Relational Model 8

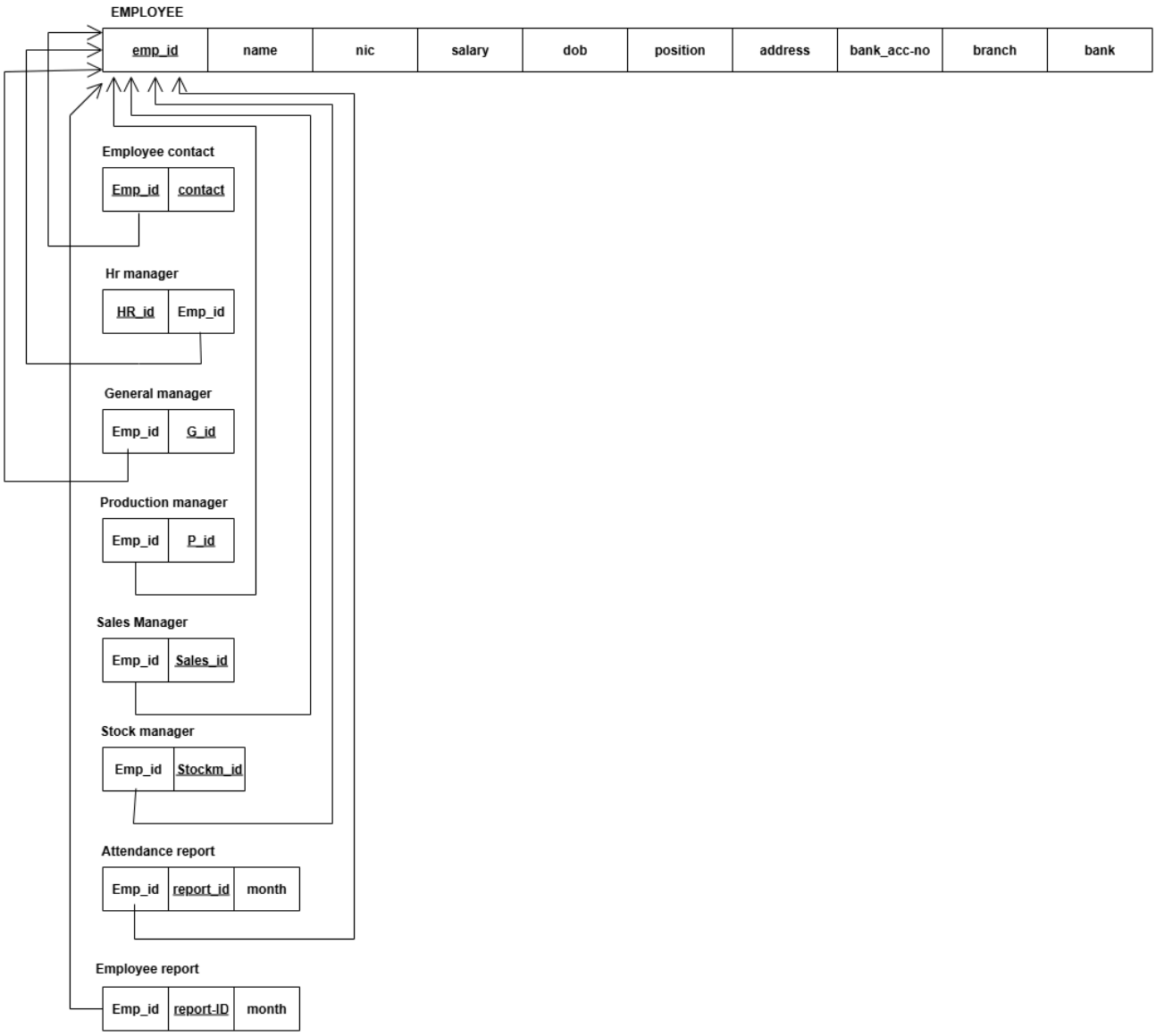
3. Database Implementation 12

1.ER diagram

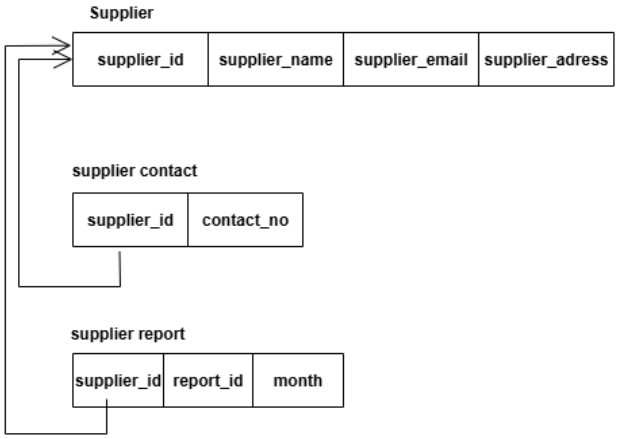


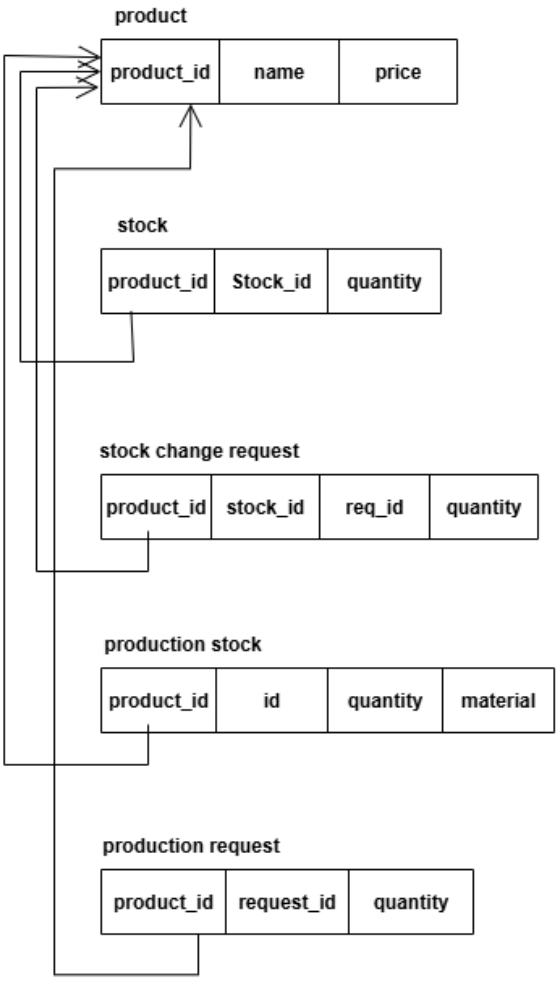
Link - [ER Diagram]

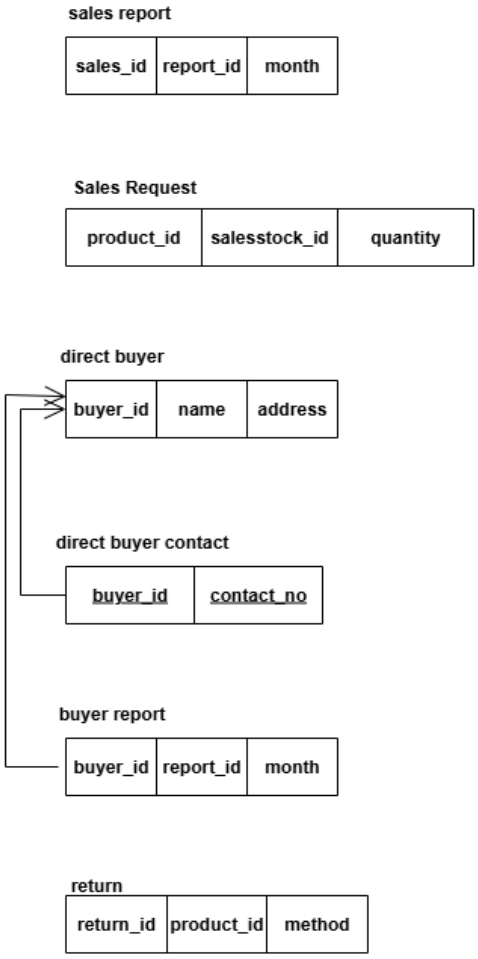
2. Relational Model

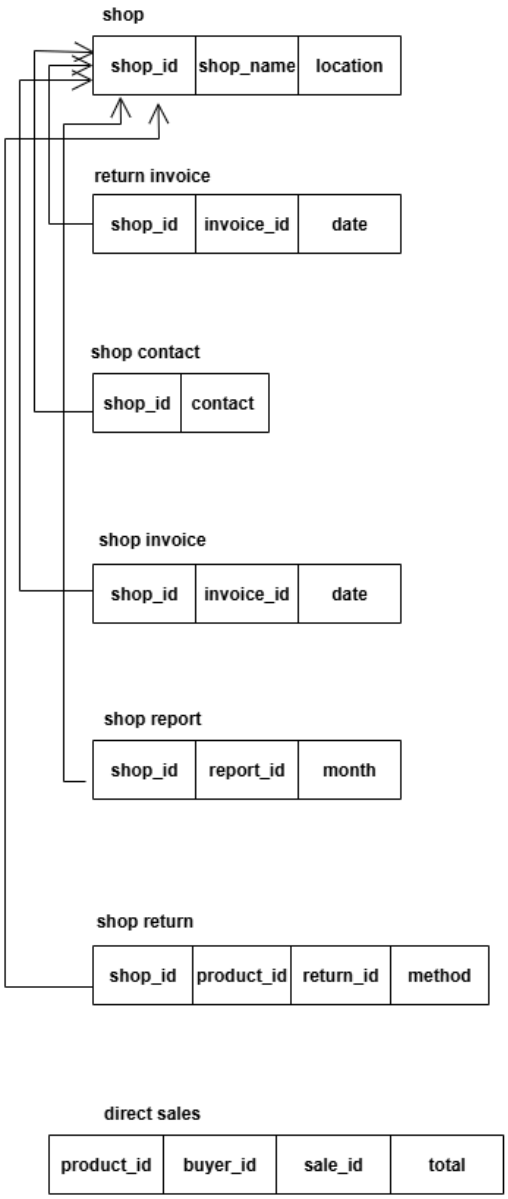


Link -[Whole Relational Diagram]


Supplier
supplier_id
supplier_name
supplier_email
supplier_adress
supplier contact
supplier_id
contact_no
supplier report
supplier_id
report_id
month


product
product_id
name
price
stock
product_id
Stock_id
quantity
stock change request
product_id
stock_id
req_id
quantity
production stock
product_id
id
quantity
material
production request
product_id
request_id
quantity


sales report
sales_id
report_id
month
Sales Request
product_id
salesstock_id
quantity
direct buyer
buyer_id
name
address
direct buyer contact
buyer_id
contact_no
buyer report
buyer_id
report_id
month
return
return_id
product_id
method


shop
shop_id
shop_name
location
return invoice
shop_id
invoice_id
date
shop contact
shop_id
contact
shop invoice
shop_id
invoice_id
date
shop report
shop_id
report_id
month
shop return
shop_id
product_id
return_id
method
direct sales
product_id
buyer_id
sale_id
total

3.Database Implementation

a) Attendance

```
// server/src/models/Attendance.Model.js
import mongoose from "mongoose";

const attendanceSchema = new mongoose.Schema({
  date: { type: Date, required: true },
  records: [
    {
      employeeId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Employee",
        required: true,
      },
      status: { type: String, enum: ["Present", "Absent"], required: true },
    },
  ],
});

const Attendance = mongoose.model("Attendance", attendanceSchema);

export default Attendance;
```

b) Direct Buyer

```
import mongoose from "mongoose";
const Schema = mongoose.Schema;

const buyerSchema = new Schema({
  name: {
    type: String,
    required: true,
  },
  contact: {
    type: Number,
    required: true,
  },
  address: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    required: true,
  },
});
const Return = mongoose.model("Buyer", buyerSchema);

export default Return;
```

c) Direct Invoice

```
import mongoose from "mongoose";
const Schema = mongoose.Schema;

const pdfSchema = new Schema({
    pdf:{
        type:String,
        required:true,
    },
    title:{
        type:String,
        required:true,
    },

});

export default mongoose.model("InvoiceDetails", pdfSchema);
```

d) Direct Returns

```
import mongoose from "mongoose";
const Schema = mongoose.Schema;

const ReturnsSchema = new Schema({
  returnId: { type: String, required: true, unique: true },
  buyerId: { type: String, required: true },
  items: [
    {
      itemName: String,
      quantity: Number,
      price: Number,
      total: Number,
    },
  ],
  totalAmount: { type: Number, required: true },
  date: { type: Date, default: Date.now },
  returnMethod: { type: String, required: true },
});

const Returns = mongoose.model("Returns", ReturnsSchema);

export default Returns;
```

e) Direct Sales

```
import mongoose from "mongoose";
const Schema = mongoose.Schema;

const SaleSchema = new Schema({
  invoiceId: { type: String, required: true, unique: true },
  buyerId: { type: String, required: true },
  items: [
    {
      itemName: String,
      quantity: Number,
      price: Number,
      total: Number,
    },
  ],
  totalAmount: { type: Number, required: true },
  date: { type: Date, default: Date.now },
});

const Sales = mongoose.model("Sales", SaleSchema);

export default Sales;
```

f) Employee

```
import mongoose from "mongoose";

const employeeSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  mobileNo: {
    type: String,
    required: true,
  },
  position: {
    type: String,
    required: true,
  },
  salary: {
    type: Number,
    required: true,
  },
  bankAccountNo: {
    type: String,
    required: true,
  },
  bank: {
    type: String,
    required: true,
  },
  branch: {
    type: String,
```

```
    required: true,
  },
  nicNo: {
    type: String,
    required: true,
    unique: true, // Enforce unique NIC numbers
  },
  dateOfBirth: {
    type: String,
    required: true,
  },
  address: {
    type: String,
    required: true,
  },
});

const Employee = mongoose.model("Employee", employeeSchema);

export default Employee;
```

g) Indirectbuyer

```
import mongoose from "mongoose";

const indirectbuyerSchema = new mongoose.Schema({
  buyername: {
    type: String,
    required: true,
  },
  shopname: {
    type: String,
    required: true,
  },
  contact: {
    type: Number,
    required: true,
  },

  email: {
    type: String,
    required: true,
  },
  address: {
    type: String,
    required: true,
  },
});

const Indirectbuyer = mongoose.model("Indirectbuyer", indirectbuyerSchema);

export default Indirectbuyer;
```

h) IndirectReturns

```
import mongoose from "mongoose";
const Schema = mongoose.Schema;

const IndirectReturnsSchema = new Schema({
  indirectreturnId: { type: String, required: true, unique: true },
  returnId: { type: String, required: true, unique: true },
  buyerId: { type: String, required: true },
  items: [
    {
      itemId: String,
      itemName: { type: String, required: true },
      quantity: { type: Number, required: true, min: 1 },
      price: { type: Number, required: true, min: 0 },
      total: { type: Number, required: true, min: 0 },
    },
  ],
  totalAmount: { type: Number, required: true, min: 0 },
  date: { type: Date, default: Date.now },
  returnMethod: {
    type: String,
    required: true,
    enum: ["issueProduct", "issueMoney"],
  },
});
const IndirectReturns = mongoose.model(
  "IndirectReturns",
  IndirectReturnsSchema
);
```

```
export default IndirectReturns;
```

i) <u>indirectSales</u>

```
import mongoose from "mongoose";
const Schema = mongoose.Schema;

const indirectSaleSchema = new Schema(
  {
    invoiceId: {
      type: String,
      required: true,
      unique: true,
      default: () => `INV-${Date.now()}-${Math.floor(Math.random() * 1000)}`,
    },
    buyerId: { type: String, required: true },
    items: [
      {
        itemName: String,
        quantity: Number,
        price: Number,
        total: Number,
      },
    ],
    totalAmount: { type: Number, required: true },
    date: { type: Date, default: Date.now },
  },
  { collection: "indirect_sales" }
);

// Drop any existing indexes
indirectSaleSchema.indexes().forEach((index) => {
  indirectSaleSchema.index(index.fields, { unique: false });
```

```
});

// Create a new index for invoiceId
indirectSaleSchema.index({ invoiceId: 1 }, { unique: true });

const indirectSales = mongoose.model("indirectSales", indirectSaleSchema);

export default indirectSales;
```

j) Material

```
//stock.model.js
import mongoose from "mongoose";
const materialSchema = new mongoose.Schema(
  {
    invoice_id: {
      type: String,
      required: true,
    },
    material_name: {
      type: String,
      required: true,
    },
    supplier_name: {
      type: String,
      required: true,
    },
    material_quantity: {
      type: Number,
      required: true,
    },
    lot_price: {
      type: Number,
      required: true,
    },
  },
  {
    timestamps: true,
  }
);
```

```
const Material= mongoose.model("Material", materialSchema);
export default Material;
```

k) MaterialRequest

```
import mongoose from "mongoose";

const materialRequestSchema = new mongoose.Schema(
  {
    ingredient_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Material",
      required: true,
    },
    request_quantity: {
      type: Number,
      required: true,
    },
    status: {
      type: String,
      enum: ["pending", "approved", "rejected"],
      default: "pending",
    },
  },
  {
    timestamps: true,
  }
);

const MaterialRequest= mongoose.model(
  "MaterialRequest",
  ingredientRequestSchema
);
```

```
export default MaterialRequest;
```

l) Product

```
//product.model.js
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    product_name: {
      type: String,
      required: true,
    },

    product_price: {
      type: Number,
      required: true,
    },
  },
  {
    timestamps: true,
  }
);

const Product = mongoose.model("Product", productSchema);

export default Product;
```

m)ProductionRequest

```
import mongoose from 'mongoose';

const productionRequestSchema = new mongoose.Schema({
  itemName: {
    type: String,
    required: true,
    trim: true
  },
  quantity: {
    type: Number,
    required: true,
    min: 0
  },
  unit: {
    type: String,
    required: true,
    enum: ['kg', 'ton', 'gram', 'piece', 'packet'],
    default: 'kg'
  },
  dateNeeded: {
    type: Date,
    required: true
  },
  urgency: {
    type: String,
    enum: ['low', 'medium', 'high', 'critical'],
    default: 'medium'
  },
  notes: {
```

```
    type: String,
    trim: true
  },
  managerId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  }
}, {
  timestamps: true
});

const ProductionRequest = mongoose.model('ProductionRequest',
productionRequestSchema);

export default ProductionRequest; // Changed from module.exports
```

n) SalesRequest

```
import mongoose from "mongoose";

const salesRequestSchema = new mongoose.Schema(
  {
    product_name: {
      type: String,
      required: true,
    },
    requested_quantity: {
      type: Number,
      required: true,
    },
    created_by: {
      type: String,
      required: true, // Username from login session
    },
    status: {
      type: String,
      enum: ["Pending", "Approved", "Rejected"],
      default: "Pending",
    },
  },
  {
    timestamps: true,
  }
);

const SalesRequest = mongoose.model("SalesRequest", salesRequestSchema);
```

```
export default SalesRequest;
```

o) SalesStock

```
import mongoose from "mongoose";

const salesstockSchema = new mongoose.Schema(
  {
    sp_name: {
      type: String,
      required: true,
    },
    sp_quantity: {
      type: Number,
      required: true,
      default: 0,
    },
    sp_price: {
      type: Number,
      required: false,
      default: 0,
    },
  },
  {
    timestamps: true,
  }
);

const SalesStock = mongoose.model("SalesStock", salesstockSchema);

export default SalesStock;
```

p) Stock

```
import mongoose from "mongoose";

const stockSchema = new mongoose.Schema(
  {
    product_name: {
      type: String,
      required: true,
    },
    product_quantity: {
      type: Number,
      required: true,
      default: 0,
    },
    product_price: {
      type: Number,
      required: false, // Optional since it defaults to 0
      default: 0,
    },
  },
  {
    timestamps: true,
  }
);

const Stock = mongoose.model("Stock", stockSchema);

export default Stock;
```

q) StockChangeRequest

```
import mongoose from "mongoose";

const stockChangeRequestSchema = new mongoose.Schema(
  {
    stock_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Stock",
      required: true,
    },
    request_type: {
      type: String,
      enum: ["update", "delete"],
      required: true,
    },
    proposed_changes: {
      type: mongoose.Schema.Types.Mixed,
    },
    reason: {
      type: String,
      required: true,
    },
    created_by: {
      type: String,
      required: true,
    },
    status: {
      type: String,
      enum: ["Pending", "Approved", "Rejected"],
      default: "Pending",
```

```
    },
    reviewed_by: {
      type: String,
    },
  },
  {
    timestamps: true,
  }
);

const StockChangeRequest = mongoose.model(
  "StockChangeRequest",
  stockChangeRequestSchema
);

export default StockChangeRequest;
```

r) Supplier

```
import mongoose from "mongoose";

const supplierSchema = new mongoose.Schema(
  {
    supplier_name: {
      type: String,
      required: true,
    },
    supplier_address: {
      type: String,
      required: true,
    },
    supplier_phone: {
      type: String,
      required: true,
    },

    supplier_email: {
      type: String,
      required: true,
    },
  },
  {
    timestamps: true,
  }
);

const Supplier = mongoose.model("Supplier", supplierSchema);
```

```
export default Supplier;
```

s) User

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  employeeName: { type: String, required: true },
  accessLevel: {
    type: String,
    enum: [
      "Executive",
      "Sales Management",
      "Production Management",
      "Employee Management",
      "Stock Management",
    ],
    required: true,
  },
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  nicNo: {
    type: String,
    required: true,
    unique: true,
    match: /^[0-9]{9}[vVxX]$|^[0-9]{12}$/, // NIC format: 9 digits +
'v'/'V'/'x'/'X' or 12 digits
  },
  mobileNo: {
    type: String,
    required: true,
    match: /^[0-9]{10}$/, // 10-digit mobile number
  },
```

```
    email: {
        type: String,
        required: true,
        unique: true,
        match: /^[^\s@]+@[^\s@]+\.[^\s@]+$/, // Basic email format
    },
    resetToken: { type: String },
    resetTokenExpiry: { type: Date },
    createdAt: { type: Date, default: Date.now },
});

const User = mongoose.model("User", userSchema);

export default User;
```